UNIVERSE AI ECOSYSTEM

FORGE V2

Implementation Plan

Universal Training, Strategy Evolution & Agent Improvement Engine

Initial training target: Oracle

Purpose: Define the architecture, implementation sequence, evaluation standards, and governance required to make Forge the reusable training and evolution engine for Universe agents.

CORE PRINCIPLE

Every improvement must have a hypothesis, a controlled experiment, measurable evidence, independent validation, and traceable approval.

Draft for architectural review | 7 November 2026

Document status and scope

This plan translates the revised Forge direction into an implementation-oriented architecture. It preserves the useful foundations of the original plan—telemetry ingestion, strategy experiments, ablation, walk-forward evaluation, challenger benchmarking, lineage, and Oracle compatibility—while removing the assumption that Forge must be limited by Oracle V1's existing strategy design.

Status: Proposed architecture for review. This document does not authorize changes to Oracle V1, live deployment, or autonomous promotion.

Important: The capabilities below are intended requirements, not claims that they are already implemented. The causes of Oracle's recent losing trades must be investigated from actual trade records rather than assumed.

1. Vision and mission

Forge is the central training, experimentation, evaluation, and evolution engine of the Universe AI ecosystem. It improves agents by investigating weaknesses, generating alternatives, testing them under controlled conditions, and producing evidence-backed candidate artifacts.

Forge must not be limited to conventional machine-learning training. Depending on the target agent, it should investigate strategies, decision policies, model configurations, parameters, workflows, features, and configurable components.

Oracle is the first major training target. The long-term objective is to support every Universe agent through reusable Forge Core capabilities and agent-specific training adapters.

Forge is successful when it demonstrates repeatable, measurable improvement without compromising incumbent integrity.

2. Universal architecture

2.1 Universal Forge Core

- Experiment contracts, hypothesis tracking, and experiment lifecycle management.
- Training orchestration, parameter search, feature and component ablation.
- Model and strategy evaluation, benchmarking, robustness and stress testing.
- Statistical validation, candidate/champion registries, and promotion governance.
- Experiment lineage, reproducibility, resource budgets, and drift detection.
- Artifact versioning, evidence storage, and deployment approval records.

2.2 Agent-specific training adapters

Each agent connects through an adapter describing trainable components, configurable parameters, training interfaces, success metrics, safety constraints, evaluation methodology, and deployment artifact format.

Adapter	Potential training scope
Oracle (first)	Trading strategies, risk methods, sizing, exits, timeframe policies, features and decision policies.
Atlas	Research methods, analysis pipelines, retrieval configurations and research quality.
Chronicle	Memory retrieval, knowledge organization, research scheduling and information quality.
Sentinel	Monitoring methods, anomaly detection and signal quality.
Genesis	Candidate-generation methods and quality of generated components.
Aegis	Validation, protection and security decision policies.

These future adapters are examples of intended scope. Their exact interfaces and evaluation metrics must be defined with each agent.

2.3 Architectural separation

Forge Core must not hardcode Oracle's nine-module StrategyGenome. Oracle-specific structures belong in the Oracle adapter and compatibility layer. This keeps Forge reusable across Universe.

3. Oracle Training Adapter — initial scope

Forge's first adapter must be able to investigate the complete Oracle trading decision process, not merely tune entry indicators.

- Strategy generation, selection and evolution.
- Market-regime detection, trend, momentum and volatility analysis.
- Entry conditions, exit conditions and signal-confidence thresholds.
- Stop-loss placement, take-profit selection and trade management.
- Position sizing, risk allocation, exposure management and portfolio constraints.
- Multi-timeframe analysis, timeframe selection, weighting and confirmation policies.
- Trade filtering, opportunity selection and strategy specialization by symbol/regime.
- Feature discovery and evaluation of new decision inputs.

3.1 Stop-loss research

Experiment with fixed-distance, ATR-based, market-structure, swing-high/low, support/resistance, supply/demand invalidation, volatility-adjusted and hybrid stop methods. Evaluate monetary risk, drawdown, stop-out frequency, net profitability and robustness.

3.2 Take-profit and exit research

Experiment with fixed reward/risk targets, structure-based targets, ATR targets, support/resistance objectives, trailing exits, partial profit-taking, dynamic targets and exits triggered when the original trade thesis becomes invalid.

3.3 Position-sizing research

Investigate fixed-fractional, volatility-adjusted, stop-distance-based, risk-budget-based, confidence-adjusted, regime-aware and portfolio-exposure-aware sizing. Position size must account for stop distance, contract specifications, account currency and permitted monetary risk.

A wider stop should generally mean a smaller position when monetary risk is held constant. A strategy must not improve apparent returns by silently increasing account risk.

3.4 Risk methods and safety boundaries

Risk-management methods are trainable. Existing V1 stop, target and sizing formulas must not be treated as permanent constraints simply because they are currently hardcoded. Forge may discover alternatives within explicit account-level safety boundaries.

- Maximum risk per trade and aggregate open risk.
- Maximum portfolio drawdown and daily/session loss.
- Maximum leverage, exposure and broker margin constraints.
- Emergency shutdown, order reconciliation and recovery requirements.
- Explicit limits on experimentation and deployment permissions.

Safety boundaries constrain the search space; they must not freeze the old V1 trading logic.

4. Multi-Timeframe Intelligence and Top-Down Analysis

Top-down multi-timeframe analysis is a primary Oracle training objective. Forge should develop and optimize a hierarchical decision process using higher-timeframe context and lower-timeframe entry information.

Timeframe	Initial research responsibility
W1 — Weekly	Broad market direction, major structure and long-term price zones.
D1 — Daily	Daily structure, support/resistance, supply/demand and regime context.
H4 — Four-hour	Intermediate trend, structural changes and developing opportunities.
H1 — One-hour	Refined directional bias and developing trade setups.
M30 — 30-minute	Entry-structure development.
M15 — 15-minute	Setup refinement, confirmation and invalidation.
M5 — 5-minute	Entry timing and execution precision.

These are initial research roles, not permanent rules. Forge must be able to test and revise them.

4.1 What Forge must learn

- Which timeframes provide useful information and which are redundant.
- Which combinations work in trending, ranging, volatile and transitional regimes.
- Whether weighted evidence is better than requiring agreement from every timeframe.
- When a lower-timeframe opportunity remains valid during a higher-timeframe pullback.
- How timeframe importance and confirmation thresholds vary by symbol and regime.
- Whether strict confirmation filters suppress too many valid opportunities.

Design rule: The goal is contextual agreement, not perfect agreement. A trade must not be rejected solely because all seven timeframes do not point in the same direction.

4.2 Experiment families

- Timeframe ablation and redundancy analysis.
- Timeframe weighting and combination searches.
- Hierarchical decision-policy experiments.
- Regime-specific timeframe selection.
- Entry-timeframe and confirmation-threshold optimization.
- Sensitivity tests for stale, missing or misaligned timeframe data.

4.3 Data synchronization

Define candle-close semantics, timezone handling, missing-bar behavior, resampling rules and prevention of look-ahead leakage. Higher-timeframe values must only become available to a historical decision after the relevant candle would actually have closed.

5. Explainable trading decisions

Oracle strategies should produce structured, testable reasons for trade proposals and no-trade decisions. Explanations must be linked to the data and rules that generated the decision; plausible text alone is not evidence.

5.1 Trade thesis

- Market context and detected regime.
- Directional bias and relevant timeframe evidence.
- Identified setup and entry justification.
- Stop-loss invalidation level and structural/volatility reason.
- Take-profit objective and expected reward relative to risk.

- Position-size calculation and monetary risk.
- Execution assumptions, spread and expected costs.
- Conditions that invalidate the thesis and trigger exit or trade management.

5.2 No-trade explanations

Distinguish no valid setup, insufficient evidence, conflicting signals, unfavourable reward/risk, unsuitable volatility, excessive spread, execution constraints, and a setup that is developing but not yet confirmed.

Forge should analyze no-trade decisions alongside executed trades. It should investigate whether filters prevent poor trades, suppress valid opportunities, or do both in different regimes.

6. Strategy experimentation and evolution

Experiment family	Purpose
Parameter sensitivity	Measure how parameter changes affect results and stability.
Module ablation	Disable a component to estimate contribution or redundancy.
Module replacement	Test alternative indicators, filters and decision components.
Strategy evolution	Generate candidates by modifying or combining validated components.
Feature discovery	Investigate whether new market features improve decisions.
Multi-timeframe optimization	Discover useful timeframe combinations and weighting policies.
Regime specialization	Test strategies tailored to identifiable market conditions.
Failure analysis	Investigate losses, stop-outs, missed opportunities and poor exits.
Robustness testing	Test whether improvements survive different periods and regimes.

Exploratory and confirmatory experiments must be separate. Candidates discovered through repeated searches must be evaluated on independent evidence not used to select them. Record the number and scope of experiments to expose multiple-testing risk.

7. Oracle telemetry and feedback loop

Forge should ingest Oracle telemetry, including strategy demo outcomes, decision observations, improvement requests and trader heartbeats. Preserve the original plan's bridge/outbox foundation.

- Receive and validate events; deduplicate with stable event identifiers.
- Preserve event time, symbol, timeframe, strategy version and decision context.
- Record outcomes and decision explanations.
- Classify recurring failure patterns and improvement opportunities.
- Create research tasks and schedule experiments when sufficient evidence exists.
- Track experiment budgets, resource limits and the evidence that triggered each run.

Not every event should launch full training. A single losing trade should be recorded and analyzed, not treated as sufficient evidence to retrain an entire strategy. Use batching, recurring-pattern detection and explicit resource budgets.

8. Training, evaluation and validation

Step	Required activity
1	Define hypothesis and expected measurable effect.
2	Select incumbent, baseline and experiment context.
3	Create an immutable experiment contract.
4	Select data, features, configuration and cost profile.
5	Execute controlled training or candidate generation.
6	Evaluate development performance.
7	Run chronological walk-forward testing.
8	Run independent out-of-sample validation.
9	Apply statistical, stress and robustness tests.
10	Benchmark candidate against incumbent.
11	Produce an evidence report with limitations.
12	Apply certification and deployment governance.

8.1 Required trading metrics

- Net return after transaction costs; maximum drawdown; profit factor.
- Sharpe and Sortino ratios; win rate; average win and average loss.
- Expectancy per trade; realized reward/risk; trade frequency and exposure.
- Consecutive losses; recovery factor; slippage and execution-cost impact.

Win rate must not be optimized in isolation. A high win rate can coexist with negative expectancy when losses are larger than wins.

8.2 Statistical and robustness requirements

- Sufficient trade count and evidence quality for the strategy and test horizon.
- Chronological data separation and prevention of train/validation/test leakage.
- Multiple rolling walk-forward windows and a final untouched test period.
- Paired comparisons where appropriate and bootstrap methods accounting for dependence in trading observations.
- Multiple-testing awareness when many candidates/configurations are searched.
- Performance analysis across regimes, symbols and relevant stress scenarios.
- Identical data, execution and cost assumptions for incumbent and candidate.

A minimum trade count is a configurable policy, not proof of statistical significance. The pilot may use 100 executed trades as a provisional screening floor, subject to review of instrument, strategy frequency and confidence requirements.

9. Transaction-cost and data integrity

Forge must not make promotion recommendations while critical execution-cost or price-basis inputs are unknown.

Required input	Evidence required
Broker/account	Verified broker, account type and pricing schedule.
Commission	Per-side or round-turn method, units, minimums and symbol rules.
Spread	Historical bid/ask data or explicit documented spread model.

Required input	Evidence required
Slippage	Measured execution evidence where available; otherwise labeled assumptions and stress range.
OHLC price basis	Evidence that bars represent bid, ask, mid or another documented basis.
Data provenance	Source, symbol mapping, interval, timezone, timestamps and adjustment policy.
Execution model	Order type, fill assumptions, latency and broker constraints.

Do not silently default commission or slippage to zero. Do not infer Yahoo or other OHLCV price basis from source name alone. Avoid double-counting spread if prices and fill model already account for it.

Blocked state: Missing critical costs, uncertain price basis, invalid data or inadequate evidence must produce BLOCKED or UNAVAILABLE status. No certification or promotion recommendation is permitted until resolved.

10. Candidate, champion and promotion governance

Forge maintains an independent candidate registry and champion history. Certification and deployment are separate decisions.

Lifecycle state	Meaning
DISCOVERED	Candidate hypothesis or artifact recorded.
EXPERIMENTING	Training or evaluation in progress.
EVALUATED	Required evaluation outputs produced.
VALIDATED	Independent evidence and required checks passed.
CERTIFIED	Candidate satisfies defined certification policy.
APPROVED FOR DEPLOYMENT	Authorized deployment decision recorded.
DEPLOYED	Candidate deployed through approved integration.

Rejected candidates retain their experiment history. Forge must never silently replace an incumbent. A passing benchmark alone does not authorize deployment.

10.1 Oracle V1 compatibility

Forge Core and its candidate model remain independent of Oracle V1's nine-module StrategyGenome. A dedicated compatibility adapter translates an approved Forge artifact into the complete V1-compatible representation, validates schema and constraints, and prepares an export.

Oracle V1 remains protected. The adapter must not silently mutate V1's champion catalog or active strategy. Deployment requires explicit authorization, versioned artifacts, rollback capability and post-deployment monitoring.

11. Implementation phases

Phase	Deliverable and completion focus
1. Core + integration	Universal Forge Core; Oracle adapter contract; reliable telemetry; catalog/incumbent snapshots; no mutation.
2. Strategy experiments	Controlled parameter experiments, module ablation, candidate generation and reproducible runs.
3. Dynamic trade management	Stop-loss, take-profit, exit, position-sizing and risk-method experiments within safety boundaries.
4. Multi-timeframe intelligence	Top-down context, timeframe ablation/weighting, regime-aware policies and synchronized data.
5. Explainability	Structured trade/no-trade reasons, failure classification and evidence-linked reports.
6. Validation	Walk-forward, independent holdout, realistic costs, statistical checks and stress testing.
7. Candidate governance	Candidate registry, lineage, champion history, certification and explicit approval.
8. V1 compatibility	Validated StrategyGenome export, explicit deployment authorization, rollback and monitoring.
9. Advanced research	Feature discovery, module replacement, strategy combination and broader evolution.
10. Other adapters	Agent-specific adapters as training contracts and evaluation methods are defined.
11. Continuous improvement	Drift detection, scheduled research, resource budgets and evidence-driven scheduling.

Each phase must be reviewed and validated before proceeding. A UI or API endpoint does not prove that the underlying experiment and evidence behavior works.

12. First deliverable: Oracle Strategy Improvement Pilot

Forge should produce useful candidates early without waiting for every advanced capability. The first pilot uses one symbol and one existing Oracle strategy, with a frozen incumbent and reproducible data.

Candidate	Research question
A — Baseline	What does the current strategy do under verified test and cost assumptions?
B — Structure-based	Does a market-structure stop and dynamic target improve outcomes?
C — ATR-based	Does volatility-adjusted stop/target logic improve robustness?
D — Hybrid	Does combining structure and volatility information improve results?
E — Sizing variant	Does an alternative sizing method improve risk-adjusted outcomes without increasing permitted risk?

Test the same candidates across controlled timeframe configurations. Do not change multiple unrelated dimensions at once in the initial comparison unless the experiment explicitly tests their interaction.

12.1 Pilot outputs

- Baseline and candidate experiment contracts.
- Candidate artifacts and complete parameter/module differences.
- Per-trade records and structured decision explanations.
- Net performance metrics after verified costs.
- Walk-forward and independent holdout reports.
- Drawdown, exposure and risk-boundary checks.
- Timeframe contribution and no-trade analysis.
- Ranked evidence report with limitations and blocked/rejected/validated status.

- No automatic change to the live incumbent.

The report of consecutive losing Oracle trades should trigger a separate diagnostic review of trade records, entry/exit reasons, market conditions, position sizes, price basis and execution costs. It does not by itself prove a particular component caused the losses.

13. Non-negotiable principles

- Oracle V1 remains protected while Forge V2 evolves independently.
- Existing working systems are not replaced without evidence and approval.
- Risk methods are trainable within explicit, non-bypassable safety boundaries.
- No candidate is certified solely on a single backtest.
- No promotion recommendation when critical data or costs are unverified.
- Every experiment is reproducible and has complete lineage.
- Certification is separate from deployment.
- Forge Core remains agent-agnostic; agent-specific assumptions belong in adapters.
- Forge measures actual improvement, not merely strategy generation or trade frequency.
- All changes are versioned and benchmarked against the previous accepted version.

14. Acceptance criteria

The initial Oracle pilot is functionally successful only when the applicable criteria below are demonstrated:

- Forge ingests and preserves Oracle telemetry without duplicate processing.
- Forge loads a frozen incumbent and its provenance.
- Forge generates at least two distinct, reproducible candidates for a declared experiment.
- Forge varies stop-loss, take-profit, position-sizing or timeframe policies through explicit experiment contracts.
- Each candidate is evaluated on equivalent data and execution assumptions.
- Per-trade decisions include structured, traceable reasons.
- Missing critical cost or data information blocks certification.
- Walk-forward and independent holdout results are recorded.
- Candidate/incumbent comparisons are reproducible.
- Certification does not automatically deploy or mutate Oracle V1.
- An approved candidate can be exported through a validated compatibility adapter.
- Failure, rejection and rollback paths are tested.

15. Open decisions before implementation

- Which symbol and incumbent strategy will be used for the first pilot?
- Which verified broker/account and historical data source define the initial cost profile?
- What account-level safety boundaries are authoritative for Oracle?
- What provisional trade-count and statistical evidence thresholds are appropriate?
- Which Oracle components expose stable training interfaces, and which need adapter work?
- What is the approved artifact and deployment workflow for V1 compatibility?
- What compute budget and experiment concurrency limits should Forge enforce?

Unknown answers must remain explicit configuration or implementation blockers. They must not be filled with invented broker values, assumed data properties or silent defaults.

16. Final vision

Forge becomes the central intelligence-training institution of Universe. It investigates how agents work, identifies weaknesses, develops alternatives, tests those alternatives and governs their adoption.

Oracle is the first application. The objective is not simply to make Oracle trade more often or achieve a higher historical score. It is to improve market understanding, opportunity selection, trade management, risk allocation, execution decisions and learning from outcomes through repeatable evidence.

The long-term standard is universal: every agent should be improvable through an appropriate Forge adapter, with domain-specific evaluation and safety governance.